

Un tunnel costruito con due chiavi

WireGuard configurato da zero su due macchine, alzato, attraversato da un ping e poi osservato sul filo — dove l'ICMP è sparito e resta solo UDP. Catturato da una coppia accesa.

Comando `qlab run vpn-lab`

VM `vpn-lab-server` 192.168.100.1 · `vpn-lab-client` 192.168.100.2

Tunnel 10.10.0.1 ↔ 10.10.0.2, UDP 51820

Esito `qlab test vpn-lab` → 5 esercizi, 28 controlli, tutti superati

1. Due macchine, e niente di configurato

```
--- server ---
ens3                UP                10.0.2.15/24 metric 100 fec0::5054:ff:fe12:3456/64
fe80::5054:ff:fe12:3456/64
ens4                UP                192.168.100.1/24 fe80::5054:ff:fe00:101/64
--- client ---
ens3                UP                10.0.2.15/24 metric 100 fec0::5054:ff:fe12:3456/64
fe80::5054:ff:fe12:3456/64
ens4                UP                192.168.100.2/24 fe80::5054:ff:fe00:102/64
```

```
$ what is running when the lab boots
inactive
wg-quick@wg0: inactive
tunnel interfaces: 0

$ but the tools are there
wireguard-tools v1.0.20210914 - https://git.zx2c4.com/wireguard-tools/
OpenVPN 2.5.11 x86_64-pc-linux-gnu [SSL (OpenSSL)] [LZO] [LZ4] [EPOLL] [PKCS11] [MH/PKTINFO] [AEAD]
built on Jul  8 2026
```

Il laboratorio fornisce gli strumenti e nessuna configurazione. È voluto: l'esercizio è la configurazione, e ce n'è pochissima. Tutto quello che segue parte da qui.

2. Prima le chiavi, e viaggiano solo le metà pubbliche

```
server public key : yx26lNwuiYiU6vrKLY5WEKyYocqityyJMI0w38DPFWk=
client public key : LZa8WmUaaTkASdU4vrT0TipmUd/1JhpUP+Z5AI5f9kg=

Only the public halves are exchanged. Neither private key leaves its machine.

$ the server config that produced (private key redacted)
[Interface]
Address = 10.10.0.1/24
ListenPort = 51820
PrivateKey = <redacted>

[Peer]
PublicKey = LZa8WmUaaTkASdU4vrT0TipmUd/1JhpUP+Z5AI5f9kg=
AllowedIPs = 10.10.0.2/32

$ wg-quick up wg0, on both
server: [#] ip link add wg0 type wireguard
server: [#] wg setconf wg0 /dev/fd/63
server: [#] ip -4 address add 10.10.0.1/24 dev wg0
server: [#] ip link set mtu 1420 up dev wg0
client: [#] ip link add wg0 type wireguard
client: [#] wg setconf wg0 /dev/fd/63
client: [#] ip -4 address add 10.10.0.2/24 dev wg0
client: [#] ip link set mtu 1420 up dev wg0
```

Ecco tutto il modello di identità di WireGuard. Ogni macchina genera una coppia di chiavi e **tiene la metà privata**; ciò che viene scambiato è la chiave pubblica, che si può tranquillamente stampare in un documento come questo.

Non ci sono certificati, non c'è un'autorità di certificazione, non c'è una negoziazione su chi siete. Un peer è una chiave pubblica con l'elenco degli indirizzi che gli è permesso usare, e il file di configurazione è abbastanza corto da leggersi per intero.

Tre campi portano il significato:

- **Address** — l'indirizzo della macchina *dentro* il tunnel. Non ha niente a che vedere con gli indirizzi di LAN della sezione 1.
- **Endpoint** — dove mandare l'UDP cifrato. Serve solo al client: il server impara l'endpoint del client dal primo pacchetto che arriva, ed è per questo che WireGuard regge client che cambiano rete.
- **AllowedIPs** — un campo con due mestieri, ed è quello che confonde. Sul traffico in uscita è una *rotta*: manda questi indirizzi a questo peer. Sul traffico in entrata è un *controllo d'accesso*: un pacchetto decifrato da questo peer che dichiara un mittente fuori dai suoi AllowedIPs viene scartato. Instradamento e autorizzazione in una riga.

`wg-quick up` fa poi a mano ciò che altrimenti scrivereste: crea l'interfaccia, carica la configurazione, aggiunge l'indirizzo, imposta un MTU di 1420 — 1500 meno il costo dell'incapsulamento.

3. Il tunnel, in piedi

```
$ ip addr show wg0      (server)
wg0                     UNKNOWN    10.10.0.1/24
10.10.0.0/24 dev wg0 proto kernel scope link src 10.10.0.1

$ ping 10.10.0.1 from the client, through the tunnel
--- 10.10.0.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2025ms
rtt min/avg/max/mdev = 0.311/0.467/0.617/0.125 ms

$ wg show on the server  (keys shortened)
interface: wg0
  public key: yx26lNwuiYiU6vrKlY5W...
  private key: (hidden)
  listening port: 51820

peer: LZa8WmUaaTkASdU4vrT0...
  endpoint: 192.168.100.2:36396
  allowed ips: 10.10.0.2/32
  latest handshake: 3 seconds ago
  transfer: 564 B received, 476 B sent
```

Tre pacchetti, nessuna perdita, sotto il millisecondo. E `wg show` riferisce ciò che conta: un **latest handshake** di pochi secondi fa, e un conteggio di byte nelle due direzioni.

Si guardi l'endpoint che il server si è annotato: `192.168.100.2:36396`. Il client non ha mai dichiarato una porta — il server l'ha imparata dai pacchetti arrivati. E non c'è nessuno stato «connesso» da riferire, perché WireGuard non ha connessioni: è UDP, fra un handshake e l'altro è senza stato, e `latest handshake` è la cosa più simile a una sessione.

4. Cosa vede la rete

```
tcpdump on the lab LAN while the client pings 10.10.0.1 through the tunnel.  
12:59:56.448653 IP 192.168.100.2.36396 > 192.168.100.1.51820: UDP, length 128  
12:59:56.448759 IP 192.168.100.1.51820 > 192.168.100.2.36396: UDP, length 128  
12:59:56.448894 IP 192.168.100.1.51820 > 192.168.100.2.36396: UDP, length 128  
12:59:57.466532 IP 192.168.100.2.36396 > 192.168.100.1.51820: UDP, length 128  
12:59:57.466597 IP 192.168.100.1.51820 > 192.168.100.2.36396: UDP, length 128  
12:59:57.466699 IP 192.168.100.1.51820 > 192.168.100.2.36396: UDP, length 128  
12:59:58.490520 IP 192.168.100.2.36396 > 192.168.100.1.51820: UDP, length 128  
12:59:58.490583 IP 192.168.100.1.51820 > 192.168.100.2.36396: UDP, length 128
```

Every packet is UDP to port 51820. The ICMP is inside, and not visible here.

Il filtro di cattura chiedeva `udp port 51820 or icmp`, sull'interfaccia che il ping attraversa davvero. Ogni singola riga è UDP, tutte della stessa lunghezza. **Di ICMP non compare nulla** — non perché sia stato filtrato, ma perché su questo filo non esiste. I ping stanno dentro il carico cifrato.

È la cosa che vale la pena vedere invece che sentirsi raccontare: un osservatore sulla LAN apprende che due indirizzi si scambiano UDP su una porta nota, quanto e quando. Non apprende nulla su *cosa* — né il protocollo, né gli indirizzi interni, né il contenuto.

Tunnel removed; the lab is back to the state the guide starts from.

5. E OpenVPN, per contrasto

Il laboratorio installa anche OpenVPN, e l'esercizio 2 lo configura con una chiave statica. Il confronto è il senso di averli entrambi: OpenVPN gira su TLS con cifrari negoziabili, richiede un'autorità di certificazione per qualsiasi cosa vada oltre una chiave statica condivisa, e la sua configurazione occupa decine di righe. WireGuard ha una sola suite crittografica e nessuna negoziazione: per questo non c'è niente da scegliere e niente da sbagliare — e per lo stesso motivo non potrà essere adattato quando quella suite invecchierà.

6. Verifica

```
Exercise 1: All 9 checks passed  
Exercise 2: All 5 checks passed  
Exercise 3: All 3 checks passed  
Exercise 4: All 5 checks passed  
Exercise 5: All 6 checks passed  
Exercises run:      5  
Exercises passed:   5  
Exercises failed:   0  
Exercises skipped:  0  
All exercises passed!
```

7. Cosa portarsi via

- Un peer è una chiave pubblica. Le chiavi private non lasciano mai la macchina che le ha generate.
- `Address` sta dentro il tunnel, `Endpoint` fuori. Tenere distinte le due cose risolve gran parte della confusione.
- `AllowedIPs` è insieme la rotta in uscita e il filtro in entrata.
- Il server non ha bisogno di sapere dov'è il client: impara l'endpoint dal traffico.
- Non esiste una connessione. `latest handshake` è come si capisce se un tunnel è vivo.
- Sul filo c'è solo UDP. Quello che viaggia dentro non è visibile da fuori.

`guide.md` del plugin porta gli esercizi: WireGuard a mano, OpenVPN con chiave statica, analisi del traffico con tcpdump e le regole di firewall attorno.